Intelligent Cognitive Alarm Platform

- Cognitive Challenges & Verification System

G. SAI SOUMYA SRI

Table of Contents

1.Project Description

Milestone 3 (Weeks 5 & 6) covers the Adaptive Intelligence & Recommendations phase of the Intelligent Cognitive Alarm Platform. My individual share of this milestone spans three connected pieces of the backend: sleep logging, the Habit Scoring Service, and the Behavioral Analytics Engine. These build directly on the alarm, challenge, and verification data completed in Milestones 1 and 2, and supply the real usage signals needed for the weighted habit score formula defined in the original project specification.

The remaining Milestone 3 scope — the machine-learning-based Adaptive Difficulty and Recommendation Engines — is owned by a teammate and is not covered in this report.

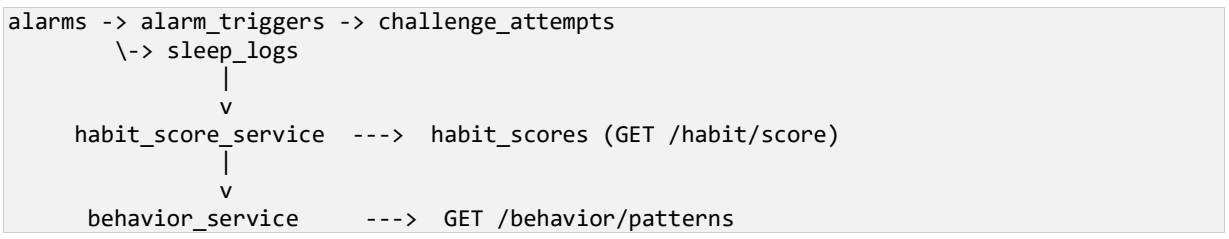
2.Environment Setup

Development continued in the same environment used for prior milestones:

- Supabase (PostgreSQL) — hosted database, all new tables verified via the Table Editor after creation through SQLAlchemy models.
- VS Code — primary IDE, on Windows.
- FastAPI / SQLAlchemy — backend framework and ORM, project structured under app/routers, app/services, app/schema, app/models.
- PowerShell (Invoke-RestMethod / Invoke-WebRequest) — used for endpoint testing in place of curl, due to PowerShell's differing curl alias.
- Expo / Expo Go — used to run the React Native mobile frontend for integration testing against the backend.

3. System Architecture / Workflow

This milestone's work sits in the Behavior Analytics Service and Habit Scoring Service layers of the overall microservices architecture, drawing on data already captured by the Alarm and Verification services:



Both services read from existing tables (alarm_triggers, challenge_attempts) plus the newly wired sleep_logs table, and write/serve their results without requiring changes to the challenge or verification services.

4. Work Completed

4.1 Sleep-Log Endpoint

A sleep-logging capability was added to give the Habit Scoring Service its missing Sleep Schedule Adherence input, using the pre-existing sleep_log.py model from the original 15-table schema design.

```
POST /sleep/log      -> app/routers/sleep.py + app/schema/sleep.py
GET  /sleep/history  -> returns a user's logged sleep sessions
```

Both endpoints were tested end-to-end against live Supabase data via PowerShell.

4.2 Habit Scoring Service

Implemented in app/services/habit_score_service.py and exposed via a rewritten app/routers/habit.py (GET /habit/score), this service calculates each user's habit score over a rolling 7-day window using the weighted formula from the original specification:

```
Habit Score =
  Wake-Up Consistency        (35%)  <- from alarm_triggers
  Challenge Completion Success (25%) <- from challenge_attempts
  Snooze Reduction            (20%)  <- from alarm_triggers.snooze_count
  Sleep Schedule Adherence    (20%)  <- from sleep_logs vs. user's target
```

This replaces the earlier placeholder scoring logic (a simple +10/-4 correct/incorrect count) used during Milestone 2, when only the Challenge Completion component had a data source.

4.3 Behavioral Analytics Engine

Implemented in app/services/behavior_service.py and exposed via a rewritten app/routers/behavior.py (GET /behavior/patterns), analyzing a 30-day window of user activity:

- Snooze trend by week — aggregated snooze_count from alarm_triggers.
- Wake-time consistency — standard deviation of actual wake times.
- Sleep-duration vs. challenge accuracy — Pearson correlation between sleep_logs duration and challenge_attempts correctness.

4.4 Issue Resolved: Schema Drift on UUID Columns

During testing, the sleep_logs, habit_scores, and goal_metrics tables were found to still have integer id columns from an earlier schema draft, since SQLAlchemy's create_all() only creates missing tables and never alters existing ones. This was resolved by dropping the three tables in the Supabase SQL Editor and restarting uvicorn (not just --reload, which only watches Python file changes, not the database) so they were recreated with the correct UUID primary keys.

5. Integration

The three new pieces built for this milestone — sleep logging, Habit Scoring, and Behavioral Analytics — were integrated into the existing FastAPI application rather than left as standalone modules:

- Wired the new sleep, habit, and behavior routers into app/main.py alongside the existing users, alarms, challenges, and verification routers, so all services run under a single FastAPI app on one port.
- Tested the integration against the mobile app itself: the React Native (Expo) frontend was run via Expo Go, connected to the FastAPI backend, and exercised end-to-end alongside the PowerShell endpoint tests.
- Added sleep_log.py and habit_score.py to the Base.metadata.create_all() import list so their tables are created automatically on app startup, consistent with the rest of the schema.

- Verified the full data path end-to-end: an alarm_trigger and challenge_attempt created via the existing Alarm/Challenge/Verification services, combined with a sleep_log entry, correctly flow into GET /habit/score and GET /behavior/patterns without any changes needed in the upstream services.
- Confirmed no regressions — re-tested /users, /alarms, /challenges, and /verify endpoints after integration to confirm they still return correctly alongside the new routers.
- Resolved the schema-drift bug (Section 4.4) as part of integration testing, since it only surfaced once the new tables were created together with the rest of the schema on a shared Supabase instance.

This integration work is what was pushed to my GitHub branch for Milestone 3 — the individual service files plus the updated main.py that ties them together.

6. Testing Summary

All three endpoint groups below were confirmed working end-to-end against real Supabase data, both individually and after integration into the shared app:

- POST /sleep/log and GET /sleep/history
- GET /habit/score
- GET /behavior/patterns
- Pre-existing endpoints (/users, /alarms, /challenges, /verify) re-verified working after integration

7. Conclusion

My individual share of Milestone 3 — sleep logging, the Habit Scoring Service, and the Behavioral Analytics Engine — is complete and tested end-to-end against live data. The habit score formula now uses three of its four real data sources (Wake-Up Consistency, Challenge Completion, Snooze Reduction, and Sleep Schedule Adherence),